COMP4502 - Final Year Project 2025-26

Final Report

Group number: gp450025020

Student Name: Ho Chun Kau, Brian

Student UID: 3036068876

Date of Submission: 02-05-2026

Abstract

Traditional AI tools for financial analysis often suffer from weak transparency, unreliable reasoning, and a tendency to produce claims without adequate support. Although large language models can generate fluent and confident-sounding responses, they do not always perform well in investment research because equity analysis requires a combination of numerical accuracy, evidence-based qualitative interpretation, and the ability to reconcile signals from multiple data sources. This project presents AIPM, a multi-agent investment research platform designed to support fundamental equity analysis through a structured, modular, and trustworthy workflow.

AIPM divides the research process into seven specialised components: business quality assessment, quantitative fundamental analysis, discounted cash flow valuation, macro context interpretation, live news monitoring, stock research from document materials, and insider activity analysis. These agents are coordinated by a central orchestration layer that interprets user queries, selects relevant analytical components, executes them in parallel, and synthesises their outputs into a structured final report.

The system is supported by a dual-database architecture with PostgreSQL managing structured financial data and Neo4j managing graph-linked textual content. A scheduled ingestion pipeline ensures data freshness without manual updates. The platform requires qualitative statements to reference retrieved source material and ensures all numerical outputs are calculated programmatically rather than generated by the language model.

AIPM demonstrates how multi-agent AI, retrieval-augmented generation, structured computation, and feedback-driven improvement can be combined into a practical financial analysis platform, delivering a more transparent, modular, and reliable alternative to conventional chatbot-style investment assistants.

Declaration & Acknowledgements

Declaration

I declare that this project is my own original work. All contributions, whether from my own efforts or from external sources, have been properly acknowledged through citations and references. I confirm that no part of this report has been previously submitted for any other course or degree at this or any other institution.

Acknowledgements

I would like to express my sincere gratitude to Dr. Chim of The University of Hong Kong for his guidance, patience, and valuable academic advice throughout the development of this Final Year Project. His constructive feedback and consistent encouragement played an important role in shaping both the technical direction and the overall presentation of this work. I am grateful for his time and support throughout every stage of the project, from initial concept to final evaluation.

I would also like to thank my project partner, Ivan, for his dedication and collaboration on this ambitious multi-agent system. The specialist agents, integration testing framework, and Docker containerisation that form the analytical backbone of this platform were primarily developed by Ivan, and his technical expertise was instrumental in bringing the system to a functioning state.

Finally, I acknowledge the support of the HKU Bloomberg Lab, where access to financial research materials enabled the construction of the document-based stock research components.

Table of Contents

Abstract *2*

Declaration & Acknowledgements *3*

Declaration **3**

Acknowledgements **3**

1. Introduction & Objectives *6*

1.1 Project Background **6**

1.2 Project Objectives **6**

1.3 Project Contributions **7**

2. Project Background and Literature Review *8*

2.1 Existing Solutions and Literature **8**

2.2 Rationale for This Project **8**

2.3 Value and Impact **9**

3. Project Methodology *9*

3.1 System Architecture Overview **9**

3.2 Data Layer: Databases and Storage Design **10**

3.2.1 PostgreSQL with pgvector **10**

3.2.2 Neo4j Graph Database **10**

3.3 Data Ingestion Pipeline **11**

3.3.1 ETL & Enrichment Stages **11**

3.3.2 Serving Layer **11**

3.4 AI Agent Layer **12**

3.4.1 Business Analyst Agent (Graph RAG) **12**

3.4.2 Quant Fundamental Agent (Non-RAG) **13**

3.4.3 Financial Modelling Agent (Non-RAG) **13**

3.4.4 Web Search Agent (Perplexity-Powered RAG) **13**

3.4.5 Stock Research Agent (Non-RAG) **13**

3.4.6 Macro Agent (Non-RAG) **14**

3.4.7 Insider News Agent (Non-RAG) **14**

3.5 Orchestration Layer **14**

3.5.1 Planner Node **14**

3.5.2 Native Parallel Fan-Out **15**

3.5.3 Per-Agent ReAct Retry Logic **15**

3.5.4 Summariser Node (Fan-In) **16**

3.5.5 Post-Processing & Self-Improving Feedback Loop **16**

3.6 User Interface: Streamlit Application **16**

3.6.1 Sidebar: API Key Configuration and Connection Testing **17**

3.6.2 Sidebar: Backend Status Panel **17**

3.6.3 Main Area: Analysis Tab **17**

3.6.4 Real-Time Agent Progress Display **18**

3.6.5 Interactive Financial Charts **18**

3.6.6 Financial Data Tables 18
3.6.7 Multi-Ticker Comparison 19
3.6.8 Citation Display and Evidence Traceability 19
3.6.9 Database Health Tab 19
3.6.10 User Feedback Capture 19
3.7 Testing Methodology 19
3.7.1 Prompt Tests (79 Tests) 20
3.7.2 Integration Tests (62 Tests) 20
3.8 User Acceptance Testing 21
4. Results & Discussion 21
4.1 Test Results 21
4.2 Infrastructure Performance 21
4.3 Agent Performance Analysis 22
4.4 Quality Assurance Evidence 22
4.5 System-Level Evaluation 22
4.6 User Acceptance Testing Results 23
5. Conclusion and Future Works 24
5.1 Conclusion 24
5.2 Future Works 24
Appendices 26
Appendix A: System Configuration 26
Appendix B: Docker Services 26
Appendix C: API Keys 26
Appendix D: Operation Manual 26
References 27
Declaration of Contribution 28

1. Introduction & Objectives

1.1 Project Background

The intersection of artificial intelligence and financial analysis has attracted considerable research attention in recent years. Large language models (LLMs) have demonstrated impressive capabilities in natural language understanding and generation, leading to their adoption in various domains including healthcare, legal research, and education. The financial services sector has been no exception — 85% of financial services companies worldwide have launched at least one AI initiative (Shalwa, 2024), reflecting the industry's recognition of AI as a transformative force. However, financial analysis presents unique challenges that are not well-addressed by general-purpose AI assistants.

Investment research requires precise numerical computation, traceable evidence, and the ability to synthesise information from diverse sources including financial statements, earnings transcripts, broker reports, market data, and macroeconomic indicators. This challenge is particularly pronounced in markets such as Hong Kong, where 2,212 SFC-licensed asset management firms collectively manage a combined AUM of US$4.5 trillion as of 2024, representing a substantial professional ecosystem with high standards for analytical rigour (Securities and Futures Commission, 2024). A single AI model asked to perform all these tasks simultaneously tends to produce outputs that sound authoritative but lack proper grounding in data. This opacity reduces trust and limits practical usefulness in a domain where the quality of reasoning matters as much as the quality of language.

1.2 Project Objectives

This project aimed to address these limitations through the development of AIPM (AI-Powered Investment Portfolio Manager), a multi-agent equity research platform. The primary objectives were as follows:

Primary Objectives:

1. **Modular Agent Architecture:** Design and implement a system where specialised agents handle distinct aspects of investment analysis, each matched to the type of reasoning and data it handles.
2. **Evidence-Grounded Output:** Ensure all qualitative claims reference retrieved source material and all numerical outputs are computed programmatically rather than generated by the language model.
3. **Orchestrated Workflow:** Build a central orchestration layer that intelligently routes queries to relevant agents, executes them in parallel, and synthesises outputs into coherent reports.
4. **Dual-Database Architecture:** Implement PostgreSQL for structured financial data and Neo4j for graph-linked textual content, enabling both numerical queries and semantic retrieval.
5. **Feedback-Driven Improvement:** Incorporate quality evaluation, user feedback collection, and episodic memory mechanisms that enable the system to improve over time.

Secondary Objectives:

1. **Accessible User Interface:** Develop a Streamlit-based web interface that makes the platform accessible to non-technical users.
2. **Containerised Deployment:** Package the entire system using Docker Compose for reproducibility and ease of deployment.
3. **Scheduled Data Ingestion:** Implement an automated pipeline that keeps databases fresh without manual intervention.

1.3 Project Contributions

The key contributions of this project are as follows:

1. **Multi-Agent Financial Research System:** A complete working platform with seven specialised agents covering business analysis, quantitative fundamentals, financial modelling, web search, stock research, macro analysis, and insider news.
2. **Evidence-Grounded Architecture:** A systematic approach to ensuring all claims in generated reports are traceable to retrieved evidence, addressing the hallucination problem that plagues general-purpose AI assistants.
3. **Coordinated Orchestration:** A LangGraph-based orchestration layer that intelligently routes queries, manages parallel execution, and synthesises outputs with proper citation.
4. **Dual-Database Design:** A practical implementation of complementary data stores that leverages the strengths of both relational and graph databases for financial analysis.
5. **Quality Assurance Framework:** An integrated evaluation system including RLAIF scoring, user feedback collection, episodic memory, and offline critic review.
6. **Accessible Interface:** A fully functional Streamlit web application that enables natural-language querying of the research platform.

1.4 Scope of the Project

This project focuses on the design, implementation, and evaluation of an automated equity research platform for publicly listed companies, with a primary emphasis on US-listed stocks accessible via the EODHD and Financial Modeling Prep (FMP) APIs. The system is scoped to fundamental and qualitative analysis — covering business analysis, quantitative fundamentals, financial modelling, macroeconomic context, web-sourced intelligence, and insider news — and does not extend to portfolio optimisation, trade execution, or real-time algorithmic trading strategies. All agents operate on data ingested through scheduled ETL pipelines and, where applicable, live web search; the system does not maintain real-time market feeds. The platform is intended as a research and decision-support tool rather than a regulated financial advisory product, and all outputs should be interpreted accordingly. Deployment is scoped to a local Docker Compose environment with optional cloud VM or ngrok-based public access, rather than a production-grade managed cloud infrastructure.

1.5 Outline of the Report

The remainder of this report is structured as follows. Part 2 reviews existing AI-based financial research solutions and the academic literature underpinning multi-agent systems and retrieval-augmented generation, before establishing the rationale and value proposition of this

project. Part 3 presents the project methodology in detail, covering the system architecture, data layer design, ingestion pipeline, all seven AI agents, the LangGraph orchestration layer, the Streamlit user interface, and the testing methodology including both automated and user acceptance testing. Part 4 presents and discusses the results, including test outcomes, infrastructure performance benchmarks, agent-level evaluation, and quality assurance evidence. Part 5 concludes the report with a summary of achievements and a discussion of directions for future work. Supporting materials including system configuration details, Docker service specifications, API key requirements, and the operation manual are provided in the Appendices.

2. Project Motivation

2.1 Existing Solutions and Literature

The application of AI to financial analysis has evolved through several phases. Early systems relied on rule-based expert systems that encoded domain knowledge into static decision trees. These were followed by statistical machine learning approaches that could identify patterns in numerical data but struggled with the textual and contextual dimensions of investment research.

The emergence of large language models marked a significant shift, enabling systems that could understand and generate natural language with unprecedented fluency. Several commercial and academic systems have explored this direction:

Commercial Investment Assistants: Bloomberg Terminal has integrated AI-powered query capabilities, and platforms such as AlphaSense and YCharts offer AI-assisted research features. These systems typically operate as retrieval front-ends that help users locate relevant content rather than performing substantive analytical reasoning.

Academic Research Systems: Academic projects have explored retrieval-augmented generation (RAG) for financial documents, demonstrating that grounding language model outputs in retrieved evidence improves factual accuracy. Studies have also examined multi-agent systems where distinct AI components handle different aspects of complex tasks.

Multi-Agent Architectures: Research in multi-agent AI systems has demonstrated that coordinating specialised agents can outperform monolithic approaches for complex tasks. This is particularly relevant for financial analysis, where different types of reasoning — numerical, textual, and contextual — require fundamentally different approaches and data sources.

2.2 Rationale for This Project

Despite these advances, significant gaps remain. Most existing solutions treat the AI system as a single-model assistant that answers questions without clearly distinguishing between calculated facts, retrieved evidence, and generated assumptions. This opacity undermines trust in a domain where investment decisions carry real financial consequences.

The multi-agent approach explored in this project addresses these limitations through five key design principles:

1. **Division of Labour:** Assigning specific tasks to specialised agents matched to the type of reasoning required.
2. **Deterministic Computation:** Ensuring all numerical outputs are calculated programmatically rather than generated by the language model.
3. **Evidence Grounding:** Requiring qualitative statements to reference retrieved source material through systematic citation.
4. **Transparency:** Making the analytical process visible through parallel execution, progress tracking, and structured output.
5. **Feedback Integration:** Building mechanisms that enable the system to learn from past successes and failures.

2.3 Value and Impact

This project demonstrates a practically viable approach to AI-powered investment research that addresses key limitations of existing solutions. The modular architecture allows for maintainability and extensibility that would be difficult to achieve in a monolithic system. The evidence-grounding approach provides a foundation for trust that is essential for practical deployment. The feedback mechanisms enable continuous improvement rather than static operation, positioning AIPM as a living system rather than a fixed artefact.

The project also carries educational value, demonstrating how multiple AI components can be coordinated into a coherent system and how database technologies can be selected and combined based on the nature of the data being stored.

3. Project Methodology

3.1 System Architecture Overview

AIPM is built around a three-layer architecture: a **data layer**, an **AI agent layer**, and an **orchestration layer**. This design reflects the understanding that real investment research involves distinct tasks requiring different tools, data sources, and reasoning approaches. The overall split of work are:

Brian's Contribution: Brian was primarily responsible for the data layer implementation, the orchestration layer development, and the user interface construction.

Ivan's Contribution: The seven specialist agents, the integration testing framework, and the Docker containerisation were primarily developed by Ivan.

3.2 Data Layer: Databases and Storage Design

The data layer serves as the foundation on which every analytical component depends. Brian implemented a dual-database architecture, with each database selected to match the nature of the data it stores.

3.2.1 PostgreSQL with pgvector

PostgreSQL serves as the primary structured data store, extended with the pgvector extension to support both relational queries and vector-based similarity search in a single system. The database schema stores the following categories of data:

- **Market Data:** Stock price history (OHLCV), trading volumes, and technical indicators
- **Financial Statements:** Income statement, balance sheet, and cash flow line items
- **Valuation Ratios:** P/E, P/B, EV/EBITDA, P/FCF, and other multiples
- **Sentiment Data:** Bullish/bearish/neutral classifications derived from news analysis
- **Insider Activity:** Transaction records including purchase and sale volumes, conviction signals
- **Institutional Ownership:** 13F filing data showing institutional holdings
- **Earnings Data:** Estimates versus actual results and surprise percentages by quarter
- **Macroeconomic Data:** US Treasury rates, yield curve shape, and S&P 500 benchmark series
- **Text Embeddings:** Vector representations of document chunks enabling semantic k-NN retrieval
- **Operational Tables:** Feedback scores, user responses, prompt versions, hallucination checks, and citation grounding records

The database was initialised through a SQL script (`docker/init-db.sql`) that creates all required tables on first startup. The schema includes appropriate indexes for common query patterns and supports vector similarity search via the pgvector extension.

3.2.2 Neo4j Graph Database

Neo4j serves as the graph and document store for company-related textual content. The graph schema includes:

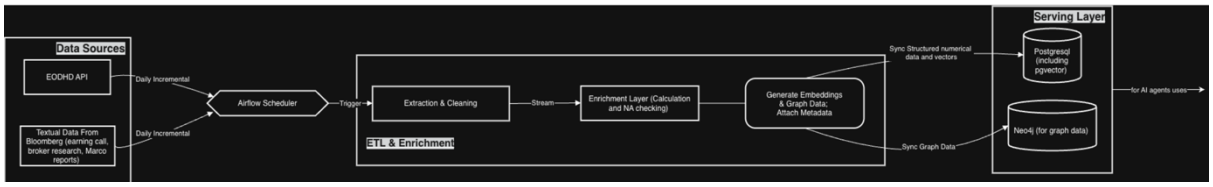
- **Company Nodes:** Rich nodes with over 85 properties each, covering financial ratios, sector classification, description fields, ESG information, and highlights
- **Document Chunks:** Text segments extracted from earnings call transcripts and broker research reports, each chunk linked to its parent document and source company
- **Relationship Edges:** Connections between companies and documents, between documents and their source materials, and between companies and their competitors

This dual-database approach avoids forcing all data into a single structure. Structured numerical queries run efficiently against PostgreSQL, while semantic and graph-linked retrieval runs against Neo4j, as illustrated in Image 1 (serving layer, right panel).

3.3 Data Ingestion Pipeline

As shown in Image 1, the data backbone consists of two incremental ingestion streams feeding into a unified ETL & Enrichment layer before reaching the serving databases.

Figure 1: Data pipeline and ETL architecture diagram.



Separating offline data preparation from the online query workflow avoids placing raw data collection inside the response path, which would increase latency and introduce unpredictable failures. The pipeline is orchestrated by **Apache Airflow** through a daily scheduled DAG configured to draw from two source streams:

- **EODHD API** — Structured, numerical market and financial data (prices, financial statements, ratios) ingested on a daily incremental basis
- **Bloomberg Textual Data** — Unstructured text: earnings call transcripts, broker research reports, and macroeconomic (Marco) reports, also ingested daily from the HKU Bloomberg Lab

3.3.1 ETL & Enrichment Stages

Stage	Description
Airflow Scheduler	Triggers both data source ingestion pipelines on configurable daily schedules, ensuring data freshness with minimal manual intervention
Extraction & Cleaning	Raw data is extracted via API or PDF scraping, then cleaned for null values, outliers, schema normalisation, and unit standardisation
Enrichment Layer (Calculation & NA Checking)	Performs derived financial calculations (e.g., EV/EBITDA, momentum metrics, margin ratios) and NA-checking/imputation to fill gaps before storage
Generate Embeddings & Attach Metadata	Text documents are chunked (1,000 characters with 200-character overlap), passed through the locally hosted Ollama model (`nomic-embed-text:v1.5`) to generate vector embeddings, and enriched with metadata (ticker, source institution, date). Graph-structured relationships are also built at this stage.

3.3.2 Serving Layer

After enrichment, data is synchronised to the dual serving layer:

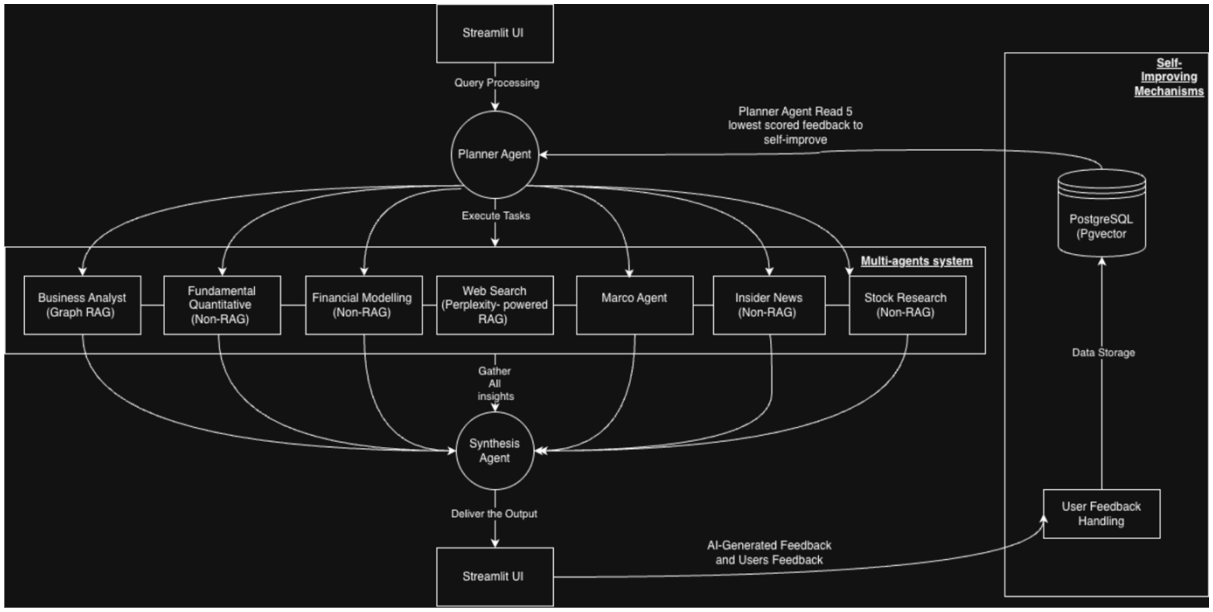
- **PostgreSQL (with pgvector)** — Receives structured numerical data and vector embeddings. Agents issue SQL queries for financial metrics and k-NN vector lookups for semantic retrieval.
- **Neo4j** — Receives graph-structured data: company nodes, document chunk nodes, and relationship edges. Used by the Business Analyst agent for multi-hop Graph RAG traversals.

After each ingestion run, a health-check script verifies that all expected tables and indexes are populated correctly, confirming that the analytical layer has reliable data before users begin querying.

3.4 AI Agent Layer

As shown in Image 2, AIPM deploys seven specialised sub-agents arranged in a parallel execution tier, each assigned to a specific domain of investment analysis. Asking one general model to perform all research tasks simultaneously reduces output quality, reduces transparency, and makes failures harder to diagnose. Ivan implemented all seven agents as his primary contribution to the project.

Image 2: AI system architecture and multi-agent orchestration diagram.



3.4.1 Business Analyst Agent (Graph RAG)

The Business Analyst agent focuses on qualitative business understanding. It retrieves evidence from Neo4j chunk nodes related to company moat, competitive positioning, management tone in earnings calls, and broker analyst commentary. It implements a **Corrective Retrieval-Augmented Generation (CRAG)** approach: after fetching relevant text, it evaluates whether the evidence is sufficiently relevant before generating a response. If retrieved material is weak (confidence below 0.4), it rewrites the query and retries. If confidence remains low (below 0.35), it falls back to a live web search. This multi-stage approach ensures qualitative analysis is grounded in source material rather than model hallucination.

Outputs: Competitive moat rating with key strengths, sentiment breakdown (bullish/bearish/neutral percentages with trend), key risk assessments with severity levels, CRAG status (`CORRECT` / `AMBIGUOUS` / `INCORRECT`), and structured document citations with grounding rate metrics.

3.4.2 Quant Fundamental Agent (Non-RAG)

The Quant Fundamental agent focuses on structured financial signal analysis. Ivan implemented deterministic Python code to compute quantitative factors, with the language model used only for narrative interpretation of already-computed scores — not to generate the numbers themselves. The agent computes a full three-axis factor model:

- **Value Factors:** P/E (trailing), EV/EBITDA, P/FCF, EV/Revenue
- **Quality Factors:** ROE, ROIC, Piotroski F-Score (0–9 financial health scale), Beneish M-Score (earnings manipulation detection)
- **Momentum & Risk:** Beta (60-day), Sharpe Ratio (12-month), 12-month return %, anomaly flags with Z-scores

Quarterly trend data (revenue, gross profit, EPS diluted, operating income) is also returned for chart rendering downstream.

3.4.3 Financial Modelling Agent (Non-RAG)

The Financial Modelling agent handles quantitative valuation. It executes a full **Discounted Cash Flow (DCF)** model with three scenarios — Bear, Base, and Bull — computing WACC, a sensitivity matrix (WACC × Terminal Growth Rate), and comparable company analysis (EV/EBITDA, P/E, P/S multiples with peer median benchmarks). A **Mixture-of-Experts (MoE) consensus** mechanism deploys three internal analyst personas — Optimist, Realist, and Pessimist — each generating independent price targets that are then aggregated into a consensus range. The agent also computes a full set of technical indicators (RSI-14, MACD, ATR, SMA-20/50/200, Bollinger Bands, Stochastic %K/%D, 52-week range) and multi-factor distress scores (Altman Z-Score, Piotroski, Beneish M).

3.4.4 Web Search Agent (Perplexity-Powered RAG)

The Web Search agent gathers current information from live sources using the **Perplexity API** as a real-time web RAG backend. This is essential because static databases and pre-ingested documents cannot capture recent events affecting market sentiment or company outlook. The agent surfaces breaking news, competitor signals, and unknown risk flags not present in offline data, and returns a structured sentiment signal (BULLISH/BEARISH/NEUTRAL) with a confidence score and sentiment rationale. A fallback mechanism is triggered and flagged in the UI if the Perplexity API is unavailable.

3.4.5 Stock Research Agent (Non-RAG)

The Stock Research agent provides deep analysis of broker research reports and earnings call transcripts stored in the PostgreSQL pgvector store. It computes **NLP signal features** on the latest transcript: KPI density (mentions per 1,000 words), hedge ratio (linguistic uncertainty), evasive language count, and a derived tone score. It enables **year-over-year transcript comparison** with period-over-period delta metrics, **Q&A behaviour analysis** to detect defensive management responses, and broker consensus rating distribution (bullish/neutral/bearish counts) with per-institution price targets. Structured citations (document name and page number) are linked back to source PDFs for evidence traceability.

3.4.6 Macro Agent (Non-RAG)

The Macro agent adds broader market context by analysing Bloomberg macroeconomic reports from the PostgreSQL/Neo4j serving layer. It identifies the current **market regime** (risk-on/risk-off), top macro themes with direction and confidence scores, top macro drivers (e.g., interest rate environment, yield curve shape), risk scenarios, and per-report summaries with stock-level relevance scores indicating how directly a macro theme affects the queried company.

3.4.7 Insider News Agent (Non-RAG)

The Insider News agent reviews insider transaction patterns and news sentiment from PostgreSQL, performing two layers of analysis:

- **Insider Activity Analysis:** Buy/sell ratio, net position classification, conviction strength, and red flag detection (e.g., cluster selling, C-suite disposals)
- **News Sentiment Analysis:** Average sentiment score across recent articles, sentiment trend direction, positive and negative catalyst identification, and a combined investment thesis with bull/bear cases and a recommendation signal

Data coverage metadata (transaction count, article count, date range) is included in the output for transparency about data recency.

Results are persisted to the operational tables in PostgreSQL for review and system improvement.

3.5 Orchestration Layer

The orchestration layer is the management intelligence of AIPM, built with **LangGraph** — a graph-based workflow framework that enables stateful, conditional, and parallel agent execution. Brian designed and implemented this component as his primary technical contribution. The compiled `StateGraph` is built once at module load time and reused across all Streamlit sessions for efficiency.

3.5.1 Planner Node

The planner node (`node_planner`) is the entry point of the graph and makes all major routing decisions before any agent is dispatched. It uses DeepSeek-Chat to perform:

- **Intent Analysis:** Classifies the user's question type (single-company analysis, multi-company comparison, sector overview, simple lookup)
- **Ticker Identification:** Resolves relevant stock symbols from natural language, including multi-company comparison queries
- **Complexity Scoring:** Assigns a complexity score of 1–3, where 1 = simple lookup (no retry, no web search), 2 = standard analysis (one retry per agent), and 3 = comprehensive deep-dive (two retries per agent, all agents enabled)

- **Agent Selection:** Sets boolean flags (`run_business_analyst`, `run_financial_modelling`, etc.) enabling or disabling specific agents based on query requirements and data availability
- **Chart Hint Generation:** Produces a `chart_hints` list that drives which Plotly visualisations are rendered in the UI
- **Episodic Memory Read:** Queries the `episodic_memory` store for the 5 lowest-scored historical interactions, injecting them as negative examples into the planner's context to guide away from known failure patterns
- **Pre-flight Checks:** Probes PostgreSQL, Neo4j, Ollama, and DeepSeek API accessibility before dispatching agents; if a service is unavailable, the affected agent is skipped gracefully rather than failing the entire pipeline

3.5.2 Native Parallel Fan-Out

Once the planner completes, the `_route_after_planner()` function returns a list of all enabled agent node names. LangGraph executes all returned nodes **simultaneously** via native fan-out, as implemented in the `StateGraph` using `add_conditional_edges` from the planner to all agent nodes. Since financial analysis subtasks are largely independent of one another, concurrent execution significantly reduces total response time compared to a sequential approach. All agents that are not enabled by the planner are silently skipped.

The fan-out targets are: `node_business_analyst`, `node_quant_fundamental`, `node_web_search`, `node_financial_modelling`, `node_stock_research`, `node_macro`, and `node_insider_news`. If the planner disables all agents (e.g., due to a malformed plan), a fallback rule always routes at least to `node_business_analyst` to prevent an empty routing list.

3.5.3 Per-Agent ReAct Retry Logic

Each agent node has its own **conditional retry edge** — a key feature of the orchestration design that prevents a single failing agent from blocking the pipeline. After each agent completes:

- If the agent produced **no output** (e.g., database connection error, LLM parse failure) **and** its iteration counter is below `react_max_iterations`, the conditional edge re-routes back to the same agent for a retry pass.
- If the agent produced output, or the maximum retry count is reached, the edge routes forward to the `summarizer` node.

The `react_max_iterations` value is set by the planner from the complexity score: complexity 1 = no retry (max = 1), complexity 2 = one retry (max = 2), complexity 3 = two retries (max = 3). Each agent independently increments its own counter in `agent_react_iterations`, so a struggling agent retries without delaying other agents that have already succeeded. The retry edge for each agent is implemented as follows (taking the Business Analyst as example):

```python
def _route_after_ba(state) -> str:
    iters = state.get("agent_react_iterations", {}).get("business_analyst", 0)
```

```
react_max = state.get("react_max_iterations") or 1
if not state.get("business_analyst_outputs") and iters < react_max:
    return "node_business_analyst"   # retry
return "summarizer"              # proceed
```

3.5.4 Summariser Node (Fan-In)

LangGraph's fan-in behaviour causes the `summarizer` node to wait until **all active agent branches** have either completed or exhausted their retries. The `node_summarizer` then combines all agent outputs into a final structured research report using DeepSeek-Chat across four synthesis stages:

- **Stage 1 — Content Assembly:** Collects and normalises all agent output dictionaries into a unified data structure
- **Stage 2 — Narrative Synthesis:** Generates section-by-section prose covering Executive Summary, Financial Analysis, Valuation, Risks, and Investment Thesis, using only the agent-provided data to minimise hallucination
- **Stage 3 — Citation Population:** Traces qualitative claims back to source documents retrieved by the Business Analyst and Stock Research agents, populating structured citation records
- **Stage 4 — Translation (Optional):** If `output_language` is set (e.g., `"cantonese"` or `"mandarin"`), performs translation as the final stage inside the same summariser call, saving an extra API round-trip

For multi-ticker queries, the summariser generates comparative sections contrasting the two companies across each dimension.

3.5.5 Post-Processing & Self-Improving Feedback Loop

After the summariser, a `node_post_processing` stage performs RLAIF (Reinforcement Learning from AI Feedback) quality scoring across four dimensions:

- **Citation Completeness:** Are qualitative claims traced to source evidence?
- **Analysis Depth:** Does the report cover all relevant financial dimensions?
- **Structure Compliance:** Are all required sections present and correctly formatted?
- **Language Quality:** Is the prose coherent, professional, and free of hallucinated specifics?

Both the AI-generated quality scores and user-submitted ratings and comments (collected via the Streamlit feedback widget) are persisted to the `feedback` and `episodic_memory` tables in PostgreSQL, forming a **self-improving feedback loop**: the episodic memory is read back by the planner at the start of each future session, injecting prior failure cases as negative context to guide better routing and agent selection over time.

3.6 User Interface: Streamlit Application

Brian developed a Streamlit-based web interface (`POC/Streamlit/app.py`) as the primary user-facing component of AIPM. The interface makes the platform accessible to non-

developers by exposing the entire orchestration pipeline through natural-language input. As shown in Image 3, the interface is organised into a **sidebar control panel** and a **main two-tab area** — the Analysis tab for running queries and the Database Health tab for infrastructure inspection.

Image 3: AIPM Streamlit user interface showing the full feature set including API key configuration, connection testing, agent progress, financial charts, database inspection, and feedback collection.

3.6.1 Sidebar: API Key Configuration and Connection Testing

The sidebar serves as the platform's configuration entry point, providing two dedicated sections for credential management. The **DeepSeek API Key** field accepts the user's key as a masked password input, pre-populated from `st.session_state` on return visits. A **Test API Connection** button immediately validates the supplied key by sending a live HTTP `GET` request to `https://api.deepseek.com/v1/models` with a 10-second timeout; a `200` response sets `st.session_state["api_verified"] = True` and writes the key to `os.environ["DEEPSEEK_API_KEY"]`, while a non-200 status or network exception surfaces an explicit error badge. The interface enforces that a verified key is present before any pipeline execution — if the user attempts to run an analysis without passing the connection test, the submit flow is blocked with inline instructions guiding the user to complete verification first.

A separate **Perplexity API Key** field below the divider accepts the credential required for the Web Search agent's Perplexity Sonar calls. This section is labelled as optional: if the key is absent, the sidebar displays an informational notice ("No Perplexity key — web search agent disabled") rather than blocking usage, since the web search agent is only activated for queries that the planner routes to it. When a key is entered, it is immediately propagated to `os.environ["PERPLEXITY_API_KEY"]` for consumption by the Web Search agent node.

3.6.2 Sidebar: Backend Status Panel

Beneath the API configuration sections, the sidebar renders a live **Backend Status** panel by calling `orchestration.data_availability.check_all()` on first load, with results cached in `st.session_state["avail_report"]` and refreshed on demand via a **Refresh Status** button. The panel reports the reachability of three services — **PostgreSQL**, **Neo4j**, and **DeepSeek** — with green/red dot indicators. If any service is unreachable, a degradation warning is surfaced, and ticker-level coverage badges indicate which of the five supported equities (AAPL, MSFT, GOOGL, TSLA, NVDA) have full versus partial data available in the local databases. A collapsible **Details** expander shows the raw availability summary string.

3.6.3 Main Area: Analysis Tab

The Analysis tab exposes the core query interface. A set of five **example query buttons** pre-populate the text area with representative investment questions spanning valuation lookups, comprehensive analyses, and multi-ticker comparisons. The free-text area accepts natural-language investment questions, with an **Advanced Options** expander providing a ticker override dropdown (to force planner resolution to a specific equity) and an **Output**

Language selector supporting 17 languages including Cantonese, Mandarin, Japanese, and Korean — values other than English are passed to the orchestration pipeline as the `output_language` parameter, triggering Stage 4 translation in the summariser. A primary **Run Analysis** button submits the query; upon completion, the final state is persisted to `st.session_state["last_state"]` and the page reloads to display results.

3.6.4 Real-Time Agent Progress Display

The interface subscribes to the agent progress queue immediately upon generating a `session_id` and before the LangGraph stream begins, using a two-queue architecture: a **node-level output queue** (polled from a background `threading.Thread` running `orchestration.graph.stream()`) and a **per-agent progress queue** (fed by `_push_progress()` calls inside each agent node). As each agent within the parallel fan-out transitions from *started* to *done* or *error*, the UI renders a live status panel listing all active agents with elapsed time in seconds, a short excerpt of the agent's initial output, and — for agents whose nodes emit a `thinking_trace` — the last three chain-of-thought steps in real time. A 50 ms polling interval keeps the display responsive without excessive CPU consumption.

3.6.5 Interactive Financial Charts

All quantitative outputs are rendered as **Plotly-powered interactive charts** via the `charts` module, enabling users to hover over data points, zoom into time ranges, and compare values dynamically. The full default chart suite comprises 16 chart types across eight sections:

- **Price & Performance:** candlestick price history with SMA/Bollinger overlays; cumulative return vs. benchmark
- **P&L Trends:** quarterly revenue bar chart with growth overlay; gross/EBIT margin trend; diluted EPS beat/miss tracker; EBITDA trend
- **Valuation:** football-field valuation range chart; DCF Bear/Base/Bull waterfall; WACC × terminal-growth-rate sensitivity heatmap
- **Peer Comparables:** multi-metric bar chart of valuation multiples against sector peers
- **Cash Flow:** free cash flow trend
- **Technical Indicators:** RSI, MACD, and ATR sub-panels
- **Sentiment & Quality:** sentiment donut; Piotroski/Beneish/Altman factor scorecard
- **MoE Consensus:** Mixture-of-Experts analyst persona consensus chart

Chart selection is driven by `chart_hints` emitted in the planner's JSON output, avoiding visual clutter for simple queries; if the planner emits no hints, the full 16-chart suite is rendered by default. For multi-ticker queries, charts are grouped into per-ticker **tabs** using `st.tabs`.

3.6.6 Financial Data Tables

Below the chart grid, the `_render_data_tables()` function renders supplementary financial data tables directly from the Quant Fundamental agent's output. These include quarterly revenue and EPS tables (last four quarters), fiscal-year aggregates computed by summing quarterly values, and trailing P/E. Tables are rendered using `st.table` with formatted strings (e.g., `$12,345.6M`) to maintain readability without requiring client-side interaction.

3.6.7 Multi-Ticker Comparison

When the planner identifies a multi-ticker query (e.g., *"Compare MSFT vs GOOGL"*), the interface switches to a side-by-side comparison layout. Both the chart grid and agent detail cards iterate over the `state["tickers"]` list, rendering each ticker's output in a separate tab or sequential card, with positional index matching used to align agent outputs to their corresponding ticker.

3.6.8 Citation Display and Evidence Traceability

Each qualitative agent card includes a collapsible **References & citations** expander. For the Business Analyst agent, the expander displays a four-metric citation dashboard (total cited, grounded, ungrounded, and grounding rate %) sourced from the `citation_report` field, followed by a grouped document list showing document names with page numbers. For the Stock Research agent, citations are grouped by document name and listed with their associated page numbers. The citation data is populated during the summariser's Stage 3 synthesis and rendered using `st.expander`, providing investment-grade evidence traceability for all qualitative claims.

3.6.9 Database Health Tab

The **Database Health** tab provides a dedicated infrastructure inspection view. Clicking **Run Database Inspection** triggers `ingestion.etl.inspect_db.check_postgres()` and `check_neo4j()` via `redirect_stdout` capture, surfacing results in two structured expanders. The PostgreSQL expander displays key row-count metrics for the `raw_timeseries`, `financial_statements`, and `news_articles` tables as `st.metric` cards alongside the raw console output. The Neo4j expander similarly parses company node counts, chunk node counts, and embedding dimensionality from the inspection script's output. Summary health cards at the top of the tab provide an at-a-glance pass/fail status for each database.

3.6.10 User Feedback Capture

A structured feedback widget (`_render_feedback_widget`) appears at the bottom of each generated report, allowing users to submit a binary helpfulness rating (Yes/No), a multi-select issue tag list (wrong numbers, missing citations, analysis too shallow, missing sections, too long/verbose, other), and an optional free-text comment. Submitted feedback is persisted to the `feedback` table in PostgreSQL via `orchestration.feedback.store_user_feedback()` and consumed by the planner's in-context learning loop (Section 3.5.1). For external sharing and testing purposes, Brian configured **ngrok** to expose the locally hosted Streamlit application through a temporary public HTTPS URL, enabling remote access by peer UAT participants without requiring cloud infrastructure deployment.

3.7 Testing Methodology

Ivan implemented a comprehensive automated testing framework covering two main categories, organised into 141 tests in total across unit and integration levels.

3.7.1 Prompt Tests (79 Tests)

Prompt tests are fast, mock-based unit tests that validate agent logic and prompt behaviour without requiring live database or API connections. The test suite (`orchestration/test_graph_nodes.py`, `orchestration/test_react_logic.py`) covers:

- **Business Analyst prompt structure and CRAG evaluation logic:** Verifies that confidence threshold branching correctly classifies retrieved evidence as `CORRECT`, `AMBIGUOUS`, or `INCORRECT`, and that the query rewriting and web fallback paths are triggered at the correct thresholds (0.4 and 0.35)
- **Citation accuracy across all source types:** Validates that citation records are populated with correct source metadata (document name, page number, chunk text) for broker research, earnings call transcripts, web results, Neo4j graph nodes, and PostgreSQL query results
- **Hallucination guards for numerical outputs:** Tests that deterministic values (DCF WACC, EPS actuals, Piotroski F-Score) are generated through code rather than model inference, and that the model's output is rejected if it contains unsupported numerical claims
- **Planner output structure and routing logic:** Confirms that `node_planner` produces valid JSON with all required fields (`ticker`, `agents_enabled`, `complexity`, `chart_hints`), and that each complexity level maps to the correct `react_max_iterations` value
- **Summariser multi-stage synthesis behaviour:** Verifies that all four synthesis stages execute in order, that multi-ticker comparison sections are correctly structured, and that translation (Stage 4) is only invoked when `output_language` is non-null

3.7.2 Integration Tests (62 Tests)

Integration tests validate end-to-end behaviour against live services and the fully assembled LangGraph graph (`orchestration/test_graph_nodes.py`, `orchestration/test_feedback_loop.py`). These tests require active PostgreSQL, Neo4j, Ollama, and DeepSeek connections and are run in the Docker Compose environment. The suite covers:

- **Agent–database connectivity:** Verifies that each agent can successfully retrieve data from both PostgreSQL (SQL queries and pgvector k-NN lookups) and Neo4j (Cypher graph traversals), and that results match expected schema structures
- **Graph node execution:** Tests the full LangGraph fan-out from planner to agents, per-agent retry conditional edges (verifying that a simulated empty output triggers a retry up to the configured maximum), and summariser fan-in (confirming all agent outputs are aggregated before synthesis begins)
- **Infrastructure readiness checks:** Probes connection latency for all four services and verifies that all required PostgreSQL tables, pgvector indexes, and Neo4j node labels are present before test execution proceeds
- **Multi-ticker workflow handling:** Submits a two-company comparison query through the full pipeline and verifies that the final report contains distinct comparative sections for both tickers
- **RLAIF and episodic memory persistence:** Submits a scored report through `node_post_processing` and verifies that quality scores are written to the `feedback`

table and that low-scoring entries are correctly promoted to the `episodic_memory` store for planner retrieval in subsequent sessions
- **Feedback loop closure:** Confirms that a low-scoring episodic memory entry injected into a subsequent planner call causes the planner to modify its routing decision — the core validation of the self-improvement mechanism

3.8 User Acceptance Testing

User acceptance testing was conducted by inviting 10 participants to interact with the platform through the Streamlit interface, accessed via the ngrok-exposed public URL. Participants were drawn from two groups: seven Computer Science / AI / Data Science students and three general or finance-background users (including one finance and investment professional), providing a breadth of perspectives across both technical and non-technical profiles. Each participant submitted natural-language investment queries of their own choosing — covering tickers such as AAPL, NVDA, and TSLA — and subsequently completed a structured Microsoft Form covering four evaluation dimensions: (1) response speed and system reliability, (2) interface usability and intuitiveness, (3) analytical accuracy and trustworthiness of AI-generated outputs, and (4) comparative quality relative to standard GenAI-generated analysis. Open-ended fields were also included to capture UI improvement suggestions and qualitative observations on agent output strengths and weaknesses.

4. Results & Discussion

4.1 Test Results

The comprehensive testing strategy yielded strong results across all test categories. On the test run dated 29 March 2026, all 141 automated tests passed:

Test Category	Count	Pass Rate	Duration
Prompt Tests	79	100%	0.18s
Integration Tests	62	100%	361.88s
Total	**141**	**100%**	**362.06s**

This 100% pass rate demonstrates that the system not only produces outputs but produces them with verified routing, structured schemas, traceable citations, live database retrieval, and properly functioning feedback mechanisms.

4.2 Infrastructure Performance

Integration tests provided concrete measurements of system performance at the infrastructure level.

Database Connectivity:

- PostgreSQL connection time: 7.79–16.02ms

- Neo4j connection time: 0.08–0.09ms
- Neo4j query latency: 24.39–45.61ms

Service Availability:

- PostgreSQL: 6 financial tables and 3 feedback tables confirmed populated
- Neo4j: Graph data and vector index (768 dimensions) confirmed operational
- Ollama: 2 models loaded (embedding model and language model)

4.3 Agent Performance Analysis

Integration tests confirmed that all seven agents successfully retrieve and process data from their respective sources. The slowest integration tests revealed where runtime cost is concentrated:

Agent	Runtime
Insider News (PostgreSQL)	197.61s
Macro (PostgreSQL + Neo4j)	91.24s
Stock Research (Neo4j)	24.38s

These retrieval-heavy workflows represent the majority of end-to-end runtime cost, indicating that future optimisation efforts should focus on data access efficiency rather than model inference.

4.4 Quality Assurance Evidence

Prompt test results provide direct evidence of design claim validity across four dimensions. Routing behaviour tests explicitly verified agent selection logic, fallback handling, and complexity scoring. Prompt quality tests verified JSON field structure, summariser stage ordering, and schema correctness. Factual grounding tests confirmed that numerical outputs are computed rather than generated by the language model. Citation accuracy tests across all source types confirmed that qualitative claims are traceable to retrieved references.

All 79 prompt tests passed, suggesting the prompt layer is both structurally reliable and aligned with evidence-grounding goals.

4.5 System-Level Evaluation

The test results support several conclusions regarding the overall system:

1. **Functional Completeness:** The system successfully executes the full pipeline from query to final report, with all components operational and verified.
2. **Evidence Grounding:** Citation accuracy tests confirm that qualitative claims are tied to retrieved evidence and all numerical outputs are calculated rather than generated by the language model.
3. **Parallel Execution:** Integration tests confirm agents run concurrently, meaningfully reducing response time compared to sequential execution.

4. **Quality Monitoring:** RLAIF evaluation and user feedback mechanisms are operational, enabling ongoing quality assessment beyond initial deployment.
5. **Infrastructure Stability:** Database connectivity and service health checks work reliably across all test runs, supporting consistent and predictable operation.

4.6 User Acceptance Testing Results

The UAT survey collected responses from 10 participants between 26 March and 25 April 2026. Results across the four scored dimensions are summarised below.

Response Speed (1–5 scale): The mean score was 4.1/5. Six participants reported response times of 3–5 minutes or under 3 minutes, while three reported 7–10 minutes and one reported over 10 minutes. One participant rated speed 2/5 due to a query exceeding 10 minutes. No participants experienced system timeouts; three noted minor issues — a language localisation failure (Spanish and Cantonese outputs not translated as selected) and one instance of mixed LaTeX and plain-text formatting in the synthesised output.

Interface Usability (1–5 scale): The mean score was 4.2/5. The agent output sections (Business Analyst, Quant, Financial Model, Web Search) and the final synthesised summary were the most frequently cited useful UI elements. Recurring improvement suggestions included export-to-PDF functionality (cited by three participants), richer visualisations, side-by-side agent comparison, and hover-over explanations for quantitative indicators to aid non-specialist users.

Analytical Accuracy & Trustworthiness (1–5 scale): The mean score was 4.5/5, the highest-rated dimension. Strengths highlighted included reliable data sourcing, DCF valuation accuracy (one finance professional specifically noted the TSLA DCF correctly identified overvaluation driven by retail investor concentration), strong visualisation, and sentiment accuracy. Weaknesses noted included citation quality in one case and DCF assumption transparency.

Comparative Quality vs. Standard GenAI: Eight out of 10 participants rated AIPM as "much better — more comprehensive and well-reasoned" compared to a typical GenAI-generated analysis, with the remaining two rating it "roughly equivalent." No participant rated it worse than a standard GenAI output.

Overall, the UAT results demonstrate strong user acceptance across all four dimensions, with analytical depth and trustworthiness receiving the highest endorsement. The identified issues — language localisation, output formatting consistency, and PDF export — represent well-scoped directions for future improvement.

In summary, the UAT results indicate that AIPM was well-received across all evaluated dimensions, achieving a combined average score of **4.27/5** across the three quantitative dimensions. The platform's most distinctive strength, as consistently reported by participants, was its analytical depth and evidence grounding relative to standard GenAI tools — a direct validation of the project's core design objective. Response speed was acceptable to the majority of users, though latency for complex multi-agent queries remains an area for optimisation. The three reported functional issues — language localisation not applying correctly, LaTeX formatting leaking into the synthesised output, and the absence of PDF export — are minor in scope and do not affect core analytical functionality. Taken together,

the UAT findings confirm that the platform successfully delivers on its intended purpose as an accessible, evidence-grounded equity research tool for both technical and non-technical users.

---

5. Conclusion and Future Works

5.1 Conclusion

AIPM successfully demonstrates a practical and technically sound approach to AI-powered investment research that addresses key limitations of conventional chatbot-style assistants. The project delivered a functioning multi-agent platform with seven specialised agents covering the full spectrum of equity analysis, from business quality assessment through to insider activity monitoring. The evidence-grounded architecture ensures that all claims in generated reports are traceable to retrieved source material and that all numerical outputs are computed programmatically, directly addressing the hallucination problem that undermines trust in general-purpose AI assistants.

The orchestrated workflow designed and implemented by Brian intelligently routes queries, manages parallel agent execution, and synthesises outputs with proper citation and structure. The dual-database design leverages PostgreSQL for structured financial data and Neo4j for graph-linked textual content, with each technology selected to match the nature of the data it stores. The quality assurance framework — encompassing RLAIF evaluation, user feedback collection, episodic memory, and Ivan's offline critic agent — provides mechanisms for continuous improvement beyond the initial deployment. The accessible Streamlit interface developed by Brian enables natural-language querying without technical expertise, broadening the practical reach of the platform.

The 100% test pass rate across 141 automated tests, combined with user acceptance testing involving 10 peers, demonstrates that the system is both technically sound and practically usable. The modular architecture ensures long-term maintainability and extensibility, while the feedback mechanisms position AIPM as a system capable of improving over time rather than remaining static.

5.2 Future Works

Several enhancements would meaningfully strengthen the platform beyond its current state:

1. **Broader Data Coverage:** Automate ingestion from additional data vendors to expand analytical coverage beyond the five stocks currently supported (AAPL, MSFT, GOOGL, TSLA, NVDA), enabling the platform to serve a wider range of investment queries.
2. **Enhanced Feedback Integration:** Implement more sophisticated reinforcement learning approaches to incorporate user feedback at the model level rather than only at the prompt level, enabling deeper learning from operational experience.
3. **Performance Optimisation:** Address the database retrieval latency identified in the Insider News and Macro agent workflows, which currently represent the dominant share of end-to-end response time, to improve the overall user experience.

4. **Production Deployment:** Move beyond the ngrok-based demonstration approach to a proper production deployment with authentication, rate limiting, and monitoring infrastructure, making the platform suitable for broader use.
5. **Additional Agent Capabilities:** Introduce agents for specific investment themes such as ESG analysis, regulatory tracking, and competitive benchmarking, guided by demand patterns identified through user acceptance testing.
6. **Improved Multi-Ticker Support:** Enhance the comparison workflow with more sophisticated side-by-side visualisations and analytical frameworks, building on the existing multi-ticker infrastructure already implemented in the orchestration layer.

Appendices

Appendix A: System Configuration

Environment Variables:

Variable	Value
PostgreSQL	localhost:5432 (airflow/airflow)
Neo4j	localhost:7474/7687 (neo4j/SecureNeo4jPass2025!)
Ollama	localhost:11434
DeepSeek Planner Model	deepseek-chat
DeepSeek Summariser Model	deepseek-reasoner

Appendix B: Docker Services

Service	Image	Purpose
postgres	pgvector:pg15	Structured data and vector similarity search
neo4j	neo4j:5.15-community	Graph data and document storage
ollama	ollama/ollama:latest	Embedding and language model serving
airflow-webserver	Custom	DAG scheduling UI
airflow-scheduler	Custom	DAG execution
streamlit	python:3.11-slim	User interface

Appendix C: API Keys

Note: API keys are not included in this distributed report. Please refer to `docker-compose.yml` for configuration instructions and required environment variable names.

Appendix D: Operation Manual

Instructions for starting the system:

1. Navigate to the project root directory
2. Run: `docker compose up -d --build`
3. Wait for all services to reach a healthy state
4. Access the Streamlit interface at `http://localhost:8501`
5. For external access, use the ngrok URL printed in the container logs

References

Caproasia. (2025, August 6). Hong Kong SFC asset & wealth management survey 2024: Asset wealth management AUM at $4.5 trillion with 2,212 licensed firms, 53,695 headcount. https://www.caproasia.com/2025/08/07/hong-kong-sfc-asset-wealth-management-survey-2024-asset-wealth-management-aum-at-4-5-trillion-with-2212-licensed-firms-53695-headcount

Shalwa. (2024, December 11). AI in finance: Key statistics, trends 2025. ARTSMART AI. https://artsmart.ai/blog/ai-in-finance-statistics-trends/

Declaration of Contribution

This project was completed as a collaborative effort between two team members. The contributions are distributed as follows:

Component	Primary Contributor
Data Layer (PostgreSQL + Neo4j schema design)	Brian
Data Ingestion Pipeline (Airflow DAG + ETL scripts)	Brian
Orchestration Layer (LangGraph workflow)	Brian
Planner Node (routing and complexity scoring)	Brian
Summariser Node (multi-agent synthesis)	Brian
Post-Processing (RLAIF scoring, episodic memory)	Brian
Streamlit User Interface	Brian
Interactive Charts (Plotly visualisations)	Brian
ngrok External Access Configuration	Brian
Business Analyst Agent	Ivan
Quant Fundamental Agent	Ivan
Financial Modelling Agent	Ivan
Web Search Agent	Ivan
Stock Research Agent	Ivan
Macro Agent	Ivan
Insider News Agent	Ivan
Critic Agent (Offline, nightly)	Ivan
Integration Testing Framework (62 tests)	Ivan
Docker Containerisation	Ivan
User Acceptance Testing	Brian and Ivan

We confirm that the work presented in this report accurately reflects our respective contributions to the project.